Content

Introduction 7
1 REQUIREMENTS ANALYSIS 8
1.1 Project Description and Objectives 8
1.2 Stakeholder Analysis 9
1.3 User Stories with Acceptance Criteria 9
1.4 Functional Requirements 10
1.5 Non-Functional Requirements 11
1.6 Use Case Diagram 12
1.7 Main Use Case Scenario: Submit Joint Housing Application 14
2. SYSTEM ANALYSIS AND MODELING 15
2.1 BPMN Business Process Diagrams 15
2.2 Data Flow Diagrams 18
2.3 ER Diagram and Data Dictionary 21
2.4 UML Class Diagram 25
2.5 Sequence Diagrams 26
2.6 State Diagram 29
3 SYSTEM ARCHITECTURE 31
3.1 Architecture Overview and Style Justification 31
3.3 Database Strategy 32
3.4 Deployment Diagram 32
3.5 Architecture Decision Records 33
4. API DESIGN 34
4.1 API Design Principles 34
4.2 Authentication Strategy 35
4.3 REST API Endpoint Specification 35
4.4 Request and Response Examples 36
4.5 Additional API Features 37
Conclusion 39
References 40

Introduction

Modern educational institutions face growing pressure to digitize student housing processes, and roommate matching involves several interdependent concerns including compatibility assessment, connection request handling, group formation, agreement confirmation, and joint housing application submission. Without a dedicated system, these workflows rely on informal communication and manual coordination, leading to mismatches, administrative overhead, and poor student experience. This coursework presents the full analysis and design of a Roommate Matching System focused on preference-based compatibility matching, controlled student communication, roommate group coordination, and joint housing application processing.

The system serves three stakeholder groups: Students who create profiles, define compatibility preferences, browse and connect with potential roommates, exchange messages via roommate chat, form groups, confirm group agreements, and submit joint housing applications; Housing Administrators who review submitted applications and record approval or rejection decisions; and supporting services including an authentication provider for university credential verification and a notification service for automated updates on requests, invitations, and application decisions.

The work is structured across four chapters covering requirements definition, system models using six complementary notations (BPMN, Data Flow Diagrams, ER Diagram, UML Class Diagram, Sequence Diagrams, and State Diagram), system architecture with component structure and deployment model, and REST API specification.

All diagrams follow standard notations including BPMN 2.0 for business processes, Yourdon-DeMarco for data flow diagrams, UML 2.5 for structural and behavioral models, and REST/JSON conventions for API specification. The resulting design artifacts form a consistent, traceable specification that directly reflects the stated requirements and serves as a solid foundation for software implementation.

1 REQUIREMENTS ANALYSIS

1.1 Project Description and Objectives

The Roommate Matching System is an information system intended to support students in finding suitable roommates and organizing shared accommodation in a structured and efficient manner. The system formalizes the roommate selection process by introducing preference-based matching, controlled communication, and group coordination within a single digital platform.

The main objectives of the system are:

– Process organization: Structure the roommate search and selection process using defined digital workflows.
– Compatibility support: Assist students in identifying roommates with similar lifestyle preferences.
– Decision transparency: Provide clear visibility of match results, requests, and application status.
– Student autonomy: Enable students to independently manage profiles, connections, and groups.
– Administrative support: Simplify the review of joint housing applications for housing staff.

System Scope:

– Web-based platform for university students
– Administrative interface for housing administrators
– Student profile and preference management
– Compatibility-based roommate recommendation
– Connection requests and messaging functionality

– Roommate group creation and coordination
– Joint housing application submission and evaluation

1.2 Stakeholder Analysis

Stakeholders are groups or roles that interact with the system or are affected by its operation and outcomes.

– Primary stakeholders: Students who use the system to create profiles, search for roommates, communicate, form groups, and submit joint housing applications.

– Secondary stakeholders: Housing Administration staff who review applications, manage housing policies, and make accommodation decisions.

– Supporting stakeholders: IT Department responsible for system maintenance and availability, and Data Protection/Security Officers responsible for ensuring compliance with data protection regulations.

1.3 User Stories with Acceptance Criteria

– US-01: Create roommate profile
  – AC: System allows entry of required personal and lifestyle information with validation
– US-02: Edit profile information
– AC: Profile changes are saved and reflected immediately in the system
– US-03: Configure profile visibility
– AC: Visibility settings correctly restrict or allow access to profile data
– US-04: Define compatibility preferences
– AC: Preferences are stored and applied during roommate matching
– US-05: View potential roommates
– AC: List displays candidates with compatibility indicators
– US-06: Send roommate request

– AC: Request is delivered and status is visible to the sender

– US-07: Respond to roommate requests

– AC: Requests can be accepted or declined with notification sent

– US-08: Exchange messages with matched users

– AC: Messaging is available only after mutual acceptance

– US-08b: Use roommate chat

– AC: Chat interface is accessible from the matched user's profile; messages are delivered in real time and stored with timestamps; chat history is visible to both parties

– US-09: Create roommate group

– AC: Group is created and member limit is enforced

– US-10: Confirm group agreement

– AC: Agreement status updates after each member confirmation

– US-11: Submit joint housing application

– AC: Submission allowed only when all required conditions are satisfied

– US-12: Process applications (Administrator)

– AC: Application decision recorded with approval or rejection reason

1.4 Functional Requirements

– Student authentication using university credentials

– Creation and modification of student profiles

– Management of lifestyle and preference data

– Configuration of profile privacy settings

– Definition and storage of compatibility parameters

– Automated roommate matching based on preferences

– Viewing and filtering lists of potential roommates

– Sending and managing roommate connection requests

– Accepting or rejecting connection requests

– Notification delivery for requests and application updates
– Private messaging between approved connections
– Real-time roommate chat with message history and timestamps
– Creation and administration of roommate groups
– Enforcement of maximum group size constraints
– Creation and confirmation of group agreements
– Submission of joint housing applications
– Validation of application completeness
– Administrative review of submitted applications
– Approval or rejection with recorded justification

1.5 Non-Functional Requirements

– Performance: Roommate search and compatibility results generated within 2 seconds; messaging actions processed within 1 second for up to 500 concurrent users

– Scalability: System supports growth to at least 10,000 registered students with horizontal scaling capability

– Availability: System uptime of at least 99% during active academic usage periods

– Security: Secure communication via HTTPS, encrypted password storage, and protection against common web vulnerabilities (OWASP Top 10)

– Usability: Mobile-responsive interface with clear navigation; compliance with basic accessibility guidelines

– Reliability: Daily automated backups with a minimum data retention period of 30 days

– Maintainability: Centralized logging and documented system interfaces to support maintenance and debugging

– Compatibility: Support for modern web browsers (Chrome, Firefox, Safari, Edge)

1.6 Use Case Diagram

The use case diagram represents the functional scope of the Roommate Matching System and the interactions between external actors and the system.

The primary actor is the Student, who interacts with the system to manage profiles, search for compatible roommates, communicate with matches, form roommate groups, and submit joint housing applications. The secondary actor is the Housing Administrator, who interacts with the system to review submitted applications and make housing decisions.

The main use cases include profile management, roommate search, connection request handling, messaging, roommate group management, joint application submission, and application review. The use case diagram provides a high-level overview of system functionality and serves as the basis for further system modeling in subsequent chapters.

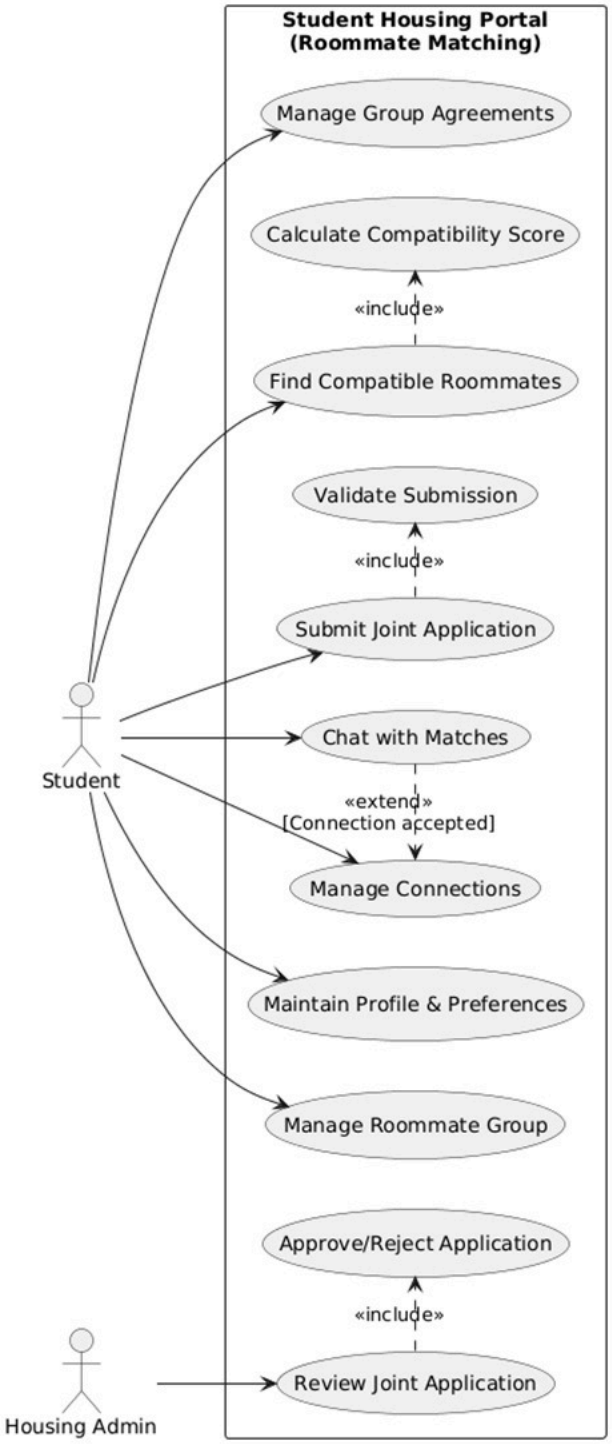


Figure 1 – Use Case Diagram

1.7 Main Use Case Scenario: Submit Joint Housing Application

Use Case ID: UC-11 – Submit Joint Housing Application

Actor: Student (group representative)

Preconditions: The student is authenticated; the roommate group has reached the required size; all group members have confirmed the group agreement.

Main Flow:

1. The student opens the group page and selects “Submit Joint Application”.
2. The system displays the application form with pre-filled group information.
3. The student selects the preferred housing option and confirms submission.
4. The system validates that all group conditions are met (group size, agreement confirmations).
5. The system records the application with status “Submitted” and assigns an application ID.
6. The system sends a confirmation notification to all group members.
7. The application is forwarded to the Housing Administrator for review.

Alternative Flow – Validation Failure: If the group has not reached the required size or not all members have confirmed, the system displays an error message and submission is blocked until the conditions are satisfied.

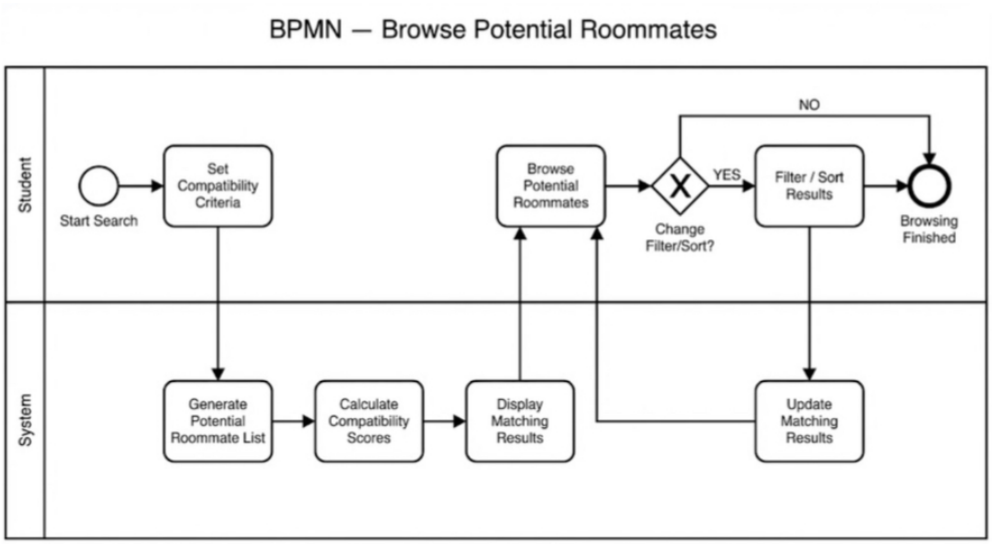
Postconditions: A joint housing application record exists with status “Submitted”; all group members have received a confirmation notification; the Housing Administrator can view and process the application.

2. SYSTEM ANALYSIS AND MODELING

2.1 BPMN Business Process Diagrams

Business Process Model and Notation (BPMN) is a standardized graphical notation used to describe business processes in information systems. BPMN allows system behavior to be represented clearly from the start of a process to its completion, showing user actions, system operations, and decision points.

In this coursework, BPMN is used to model the key processes of the Roommate Matching System based on the use cases and functional requirements defined in Chapter 1.



. Figure 2 – BPMN Diagram: Browse Potential Roommate

This BPMN diagram describes the process of searching for potential roommates. The process starts when the student begins searching and sets compatibility criteria. The system then generates a list of potential roommates, calculates compatibility scores, and displays the matching results.

The student can browse the results and optionally apply filtering or sorting. If filters or sorting are applied, the system updates the displayed results. The process ends when the student finishes browsing the final matching results.

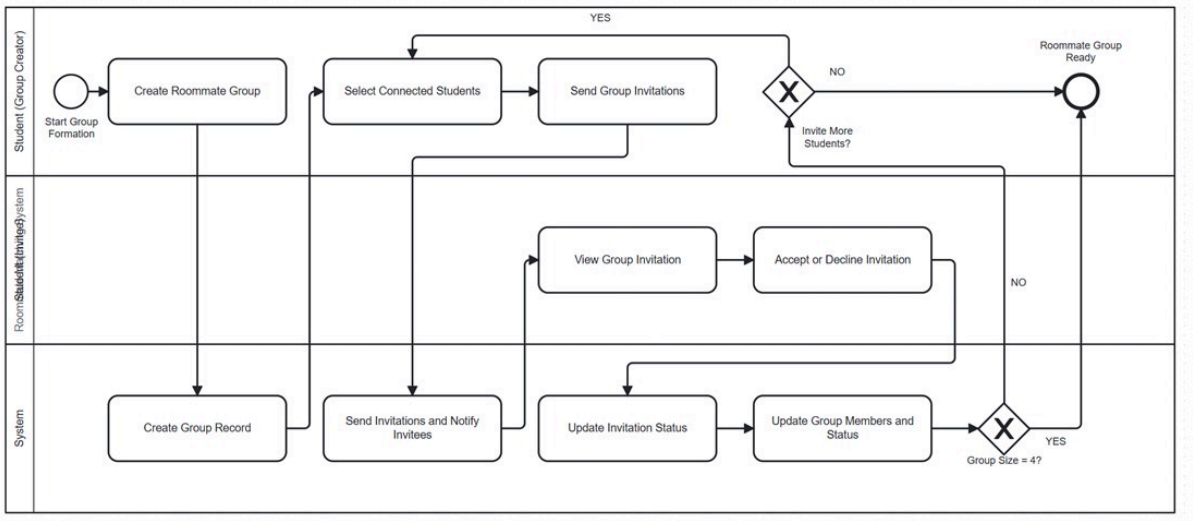


Figure 3 – BPMN Diagram: Form Roommate Group.

This BPMN diagram represents the process of creating a roommate group. The process begins when a student creates a group and selects connected students to invite. The system creates the group record and sends invitations to the selected students.

Invited students can accept or decline the invitation. The system updates the invitation status and enforces the maximum group size constraint. If the group is not complete, additional invitations can be sent. The process ends when the roommate group reaches the required size and becomes ready.

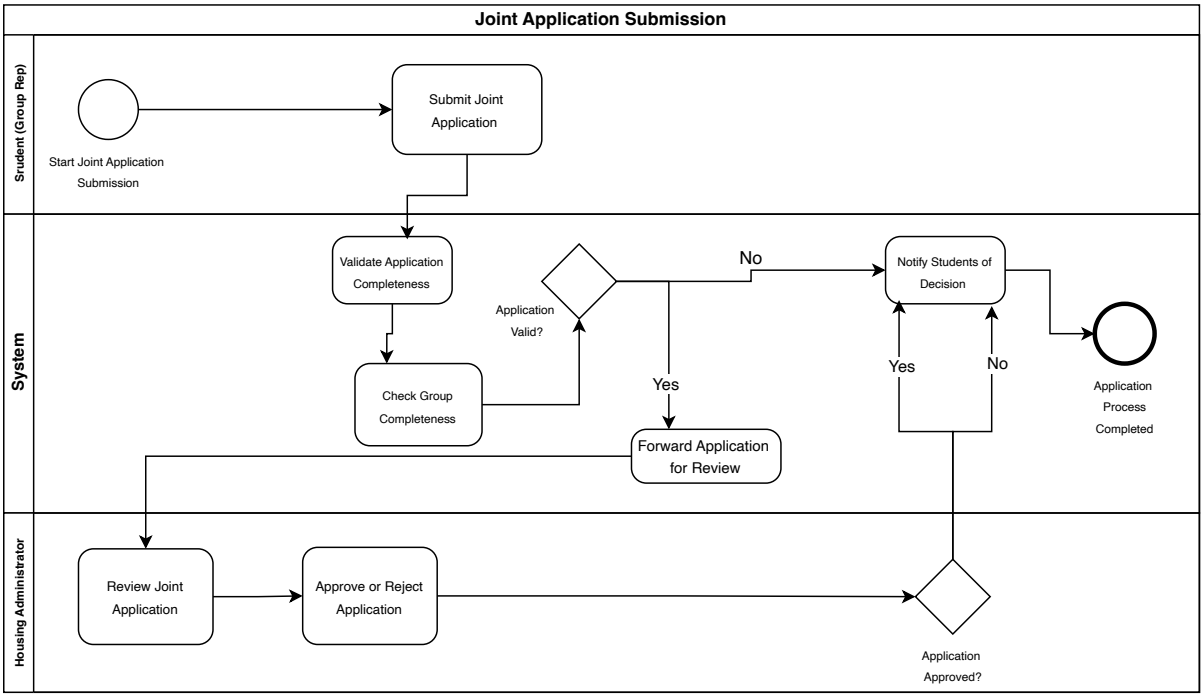


Figure 4 – BPMN Diagram: Joint Application Submission.

This BPMN diagram shows the process of submitting and reviewing a joint housing application. The process starts when a group representative submits the joint application. The system validates the application and checks group completeness before forwarding it to the housing administrator.

The administrator reviews the application and decides whether to approve or reject it. The system then notifies the students of the decision, and the process ends.

2.2 Data Flow Diagrams

Data Flow Diagrams (DFDs) are used to visually represent how data moves through a system, how it is processed, and where it is stored. Unlike UML or BPMN diagrams, DFDs focus mainly on data flow rather than user interactions or process sequences. They help in understanding the logical structure of the system and the relationships between external entities, processes, and data stores.

In this coursework, DFDs are used to model the data flow within the Roommate Matching System, showing how information such as profile data, compatibility criteria, group information, and joint application data is handled by the system.

To present the system clearly, three levels of DFDs were created**:** Level 0 (Context Diagram**),** Level 1, and Level 2. Each level provides a deeper level of detail while remaining consistent with the system requirements defined in Lab 1.

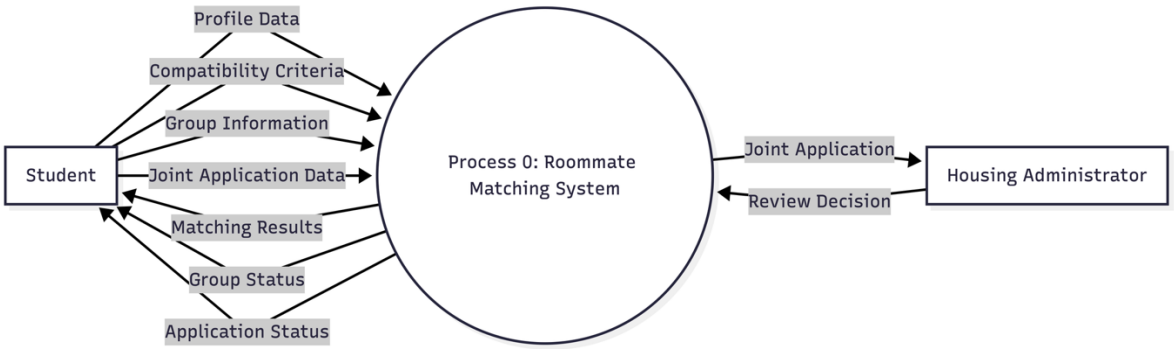


Figure 5 – Level 0 DFD (Context Diagram)

The DFD Level 0 shows the Roommate Matching System as a single process and highlights its interaction with external entities. At this level, the internal processes of the system are not shown.

The student provides inputs such as profile data, compatibility criteria, group information, and joint application data. The system returns outputs such as matching results, group status updates, and application confirmations.

The Housing Administrator interacts with the system by reviewing joint applications and managing application decisions.

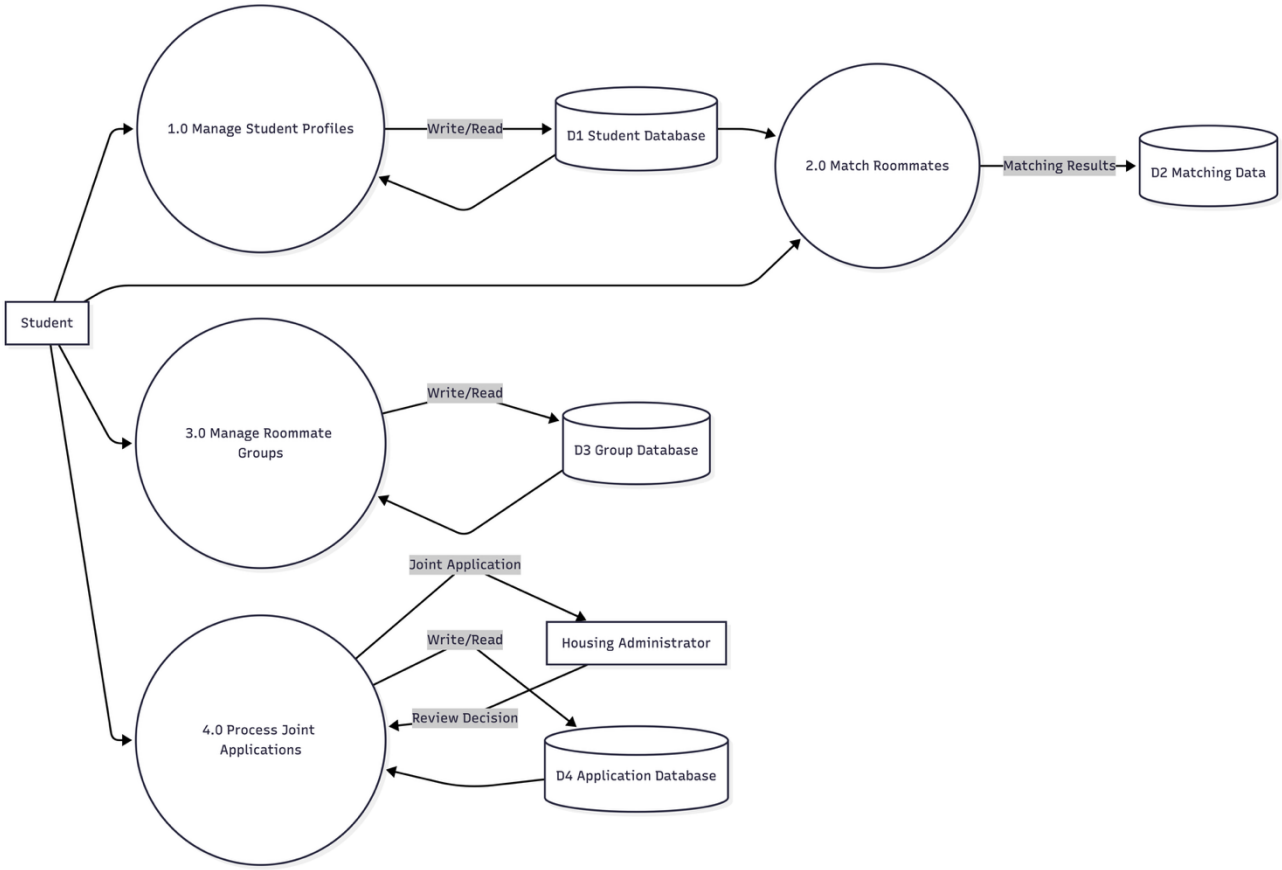


Figure 6 – Level 1 DFD

The Level 1 DFD breaks the Roommate Matching System into its main functions and shows how data flows between them.

Students first manage their profiles, and this information is stored in the student database. The system then uses this data to match roommates and generate matching results. Students can also create and manage roommate groups, with group details saved in the group database. Finally, joint applications are submitted and processed by the system, stored in the application database, and reviewed by the housing administrator.

This diagram shows how the main system processes work together and how data is stored and used across the system.

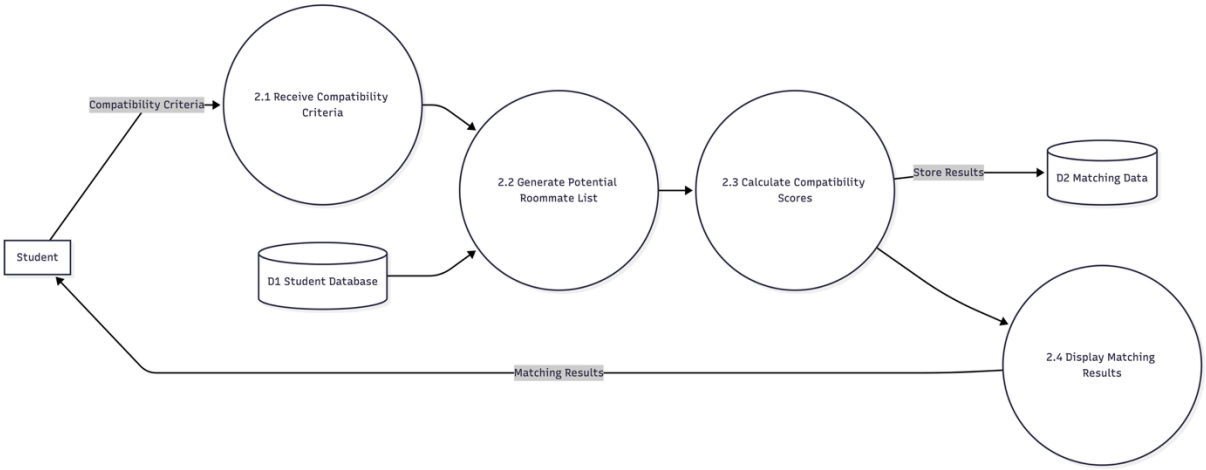


Figure 7 – Level 2 DFD

This Level 2 DFD shows how the Match Roommates process works in more detail. The student provides compatibility criteria, which the system uses to generate a list of potential roommates. The system then calculates compatibility scores, stores the results, and finally displays the matching results to the student.

This diagram explains step by step how roommate matches are created and shown to users.

2.3 ER Diagram and Data Dictionary

The Entity Relationship (ER) Diagram represents the data structure of the Roommate Matching System by showing the main entities, their attributes, and the relationships between them. It provides a clear view of how student information, compatibility criteria, roommate groups, matching results, and joint applications are

stored and connected within the database. This diagram is essential for database design, as it ensures data consistency, reduces redundancy, and supports efficient data retrieval for matching and group management processes.

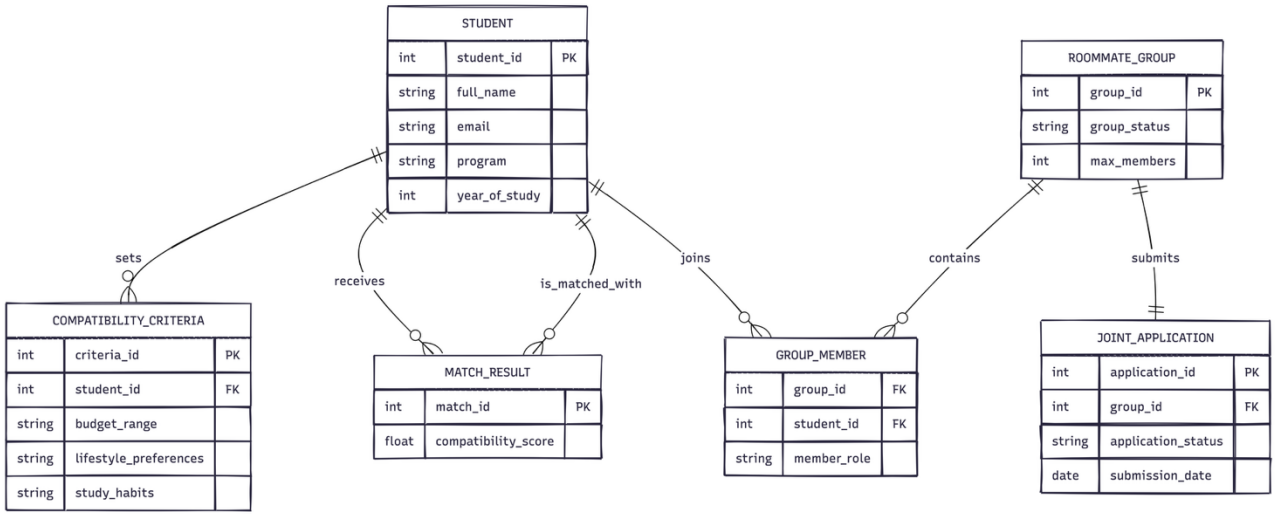


Figure 8 – ER Diagram: Database Structure

The ER diagram includes several key entities such as Student, Compatibility Criteria, Match Result, Roommate Group, Group Member, and Joint Application. These entities work together to support the core functionality of the system, including student profile management, roommate matching, group formation, and housing application processing.

Data Dictionary:

STUDENT

Attribute Name	Data Type	Description	Key
student_id	int	Unique identifier for each student	PK
full_name	string	Full name of the student	
email	string	Student email address	
program	string	Academic program of the student	
year_of_study	int	Current year of study	

COMPATIBILITY_CRITERIA

Attribute Name	Data Type	Description	Key
criteria_id	int	Unique identifier for compatibility criteria	PK
student_id	int	ID of the student who set the criteria	FK
budget_range	string	Preferred budget range for housing	
lifestyle_preferences	string	Lifestyle preferences (e.g. quiet, social)	
study_habits	string	Study habits of the student	

MATCH_RESULT

Attribute Name	Data Type	Description	Key
match_id	int	Unique identifier for a match result	PK
compatibility_score	float	Calculated compatibility score	

ROOMMATE_GROUP

Attribute Name	Data Type	Description	Key
group_id	int	Unique identifier for a roommate group	PK
group_status	string	Current status of the group	
max_members	int	Maximum number of members allowed	

GROUP_MEMBER

Attribute Name	Data Type	Description	Key
group_id	int	Identifier of the roommate group	FK
student_id	int	Identifier of the student	FK
member_role	string	Role of the student in the group	

JOINT APPLICATION

Attribute Name	Data Type	Description	Key
application_id	int	Unique identifier for the joint application	PK
group_id	int	Identifier of the submitting group	FK
application_status	string	Status of the application	
submission_date	date	Date of submission	

The data dictionary defines each entity and attribute used in the system, ensuring clarity, consistency, and accurate implementation of the database structure based on the ER diagram.

2.4 UML Class Diagram

The UML Class Diagram represents the static structure of the system by showing the main classes, their attributes, and the relationships between them. It provides a clear view of how the system's data and logic are organized and how different components interact at a high level. This diagram is useful for understanding system design and serves as a blueprint for implementing the database and application logic.

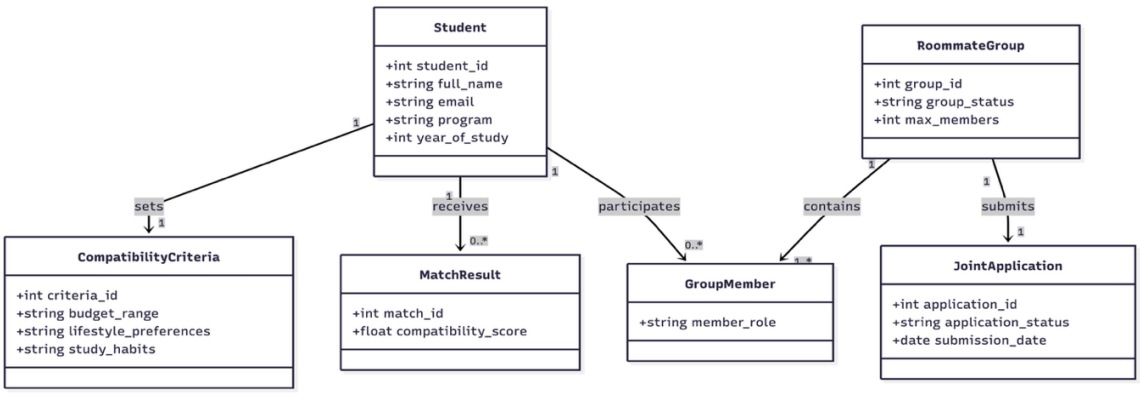


Figure – 9 UML Class Diagram

The class diagram includes core classes such as Student, CompatibilityCriteria, MatchResult, RoommateGroup, GroupMember, and JointApplication. Each class defines key attributes required by the system, while associations illustrate how students set criteria, receive match results, form groups, and submit joint housing applications.

The relationships reflect real system behavior, including one-to-many and many-to-many associations, ensuring consistency with the functional requirements and previously defined ER and DFD models.

2.5 Sequence Diagrams

Sequence diagrams show how objects interact in a particular sequence over time. They are useful for understanding the dynamic behavior of the system and identifying the order of operations in specific scenarios.

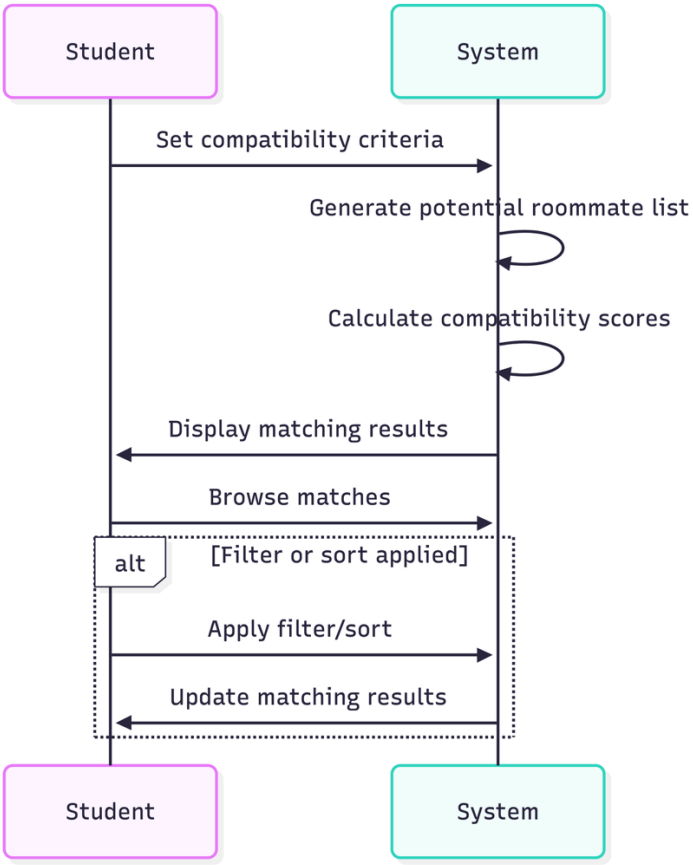


Figure 10 - Sequence Diagram 1: Browse Potential Roommates

This sequence diagram shows how a student searches for potential roommates. The process starts when the student sets compatibility criteria. The system then generates a list of potential roommates, calculates compatibility scores, and displays the results. The student can browse the results and optionally apply filters or sorting, after which the system updates the displayed matches.

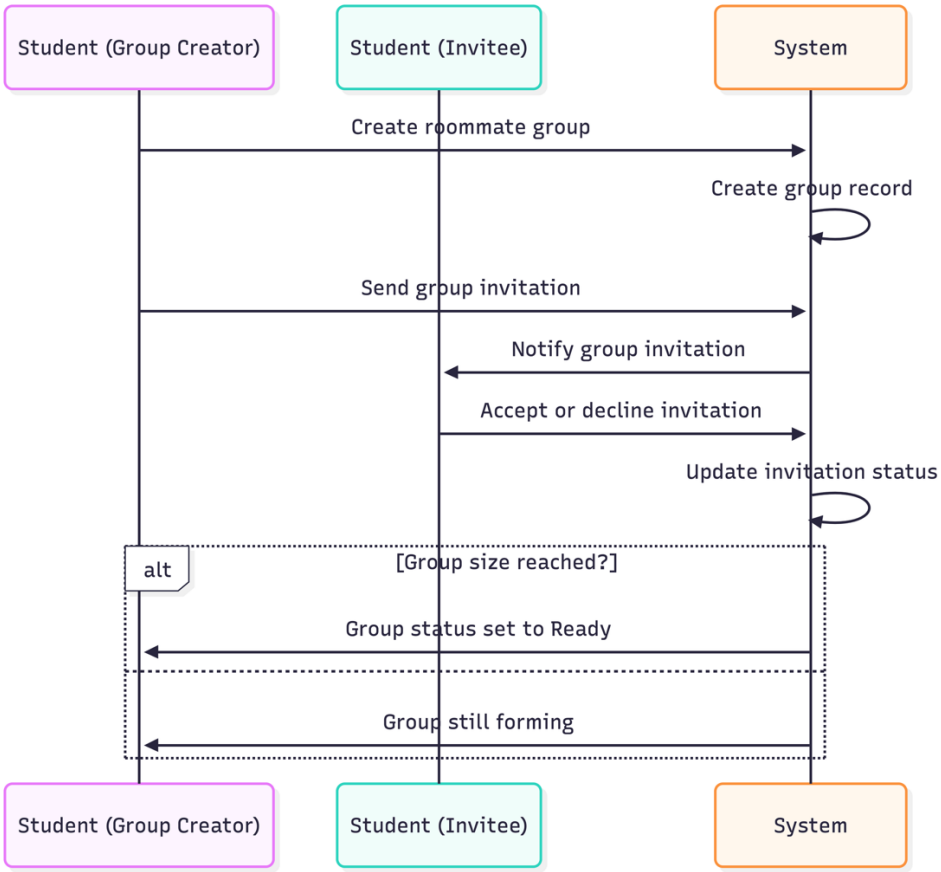


Figure 11 - Sequence Diagram 2: Form Roommate Group

This sequence diagram describes how a roommate group is formed. A student creates a group and sends invitations to connected students. Invitees can accept or decline the invitation. The system updates the group status and checks whether the maximum group size has been reached before marking the group as ready.

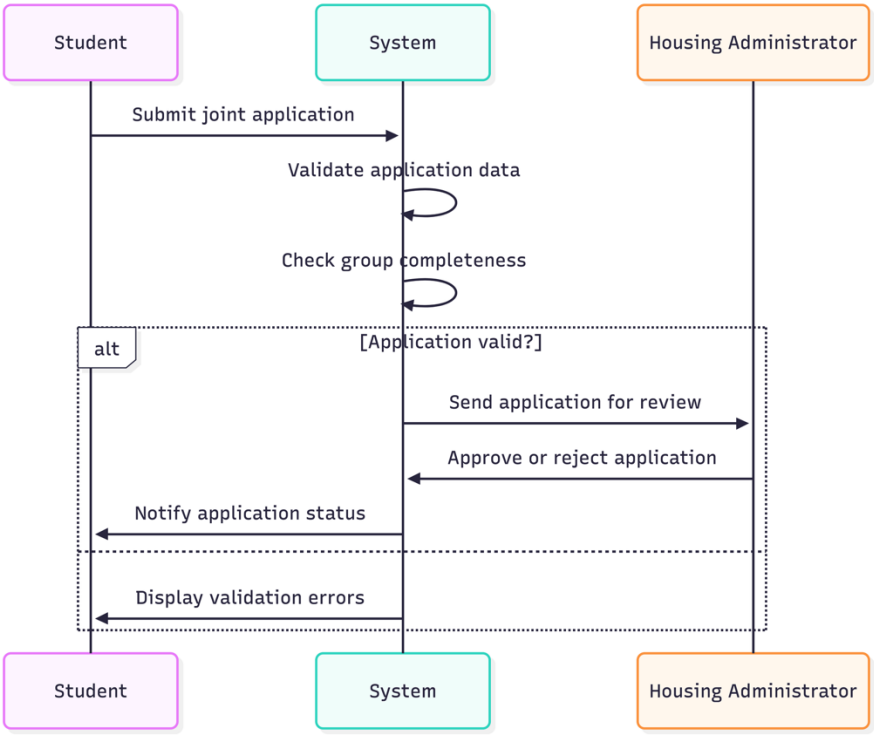


Figure 12 - Sequence Diagram 3: Submit Joint Application

This sequence diagram illustrates the joint application submission process. The student submits the application, which is validated by the system. If all group requirements are met, the application is forwarded to the housing administrator for review. The final decision is then communicated back to the student.

2.6 State Diagram

State diagrams show the different states an object can be in and the transitions between those states. They are particularly useful for modeling the lifecycle of objects with complex state behavior.

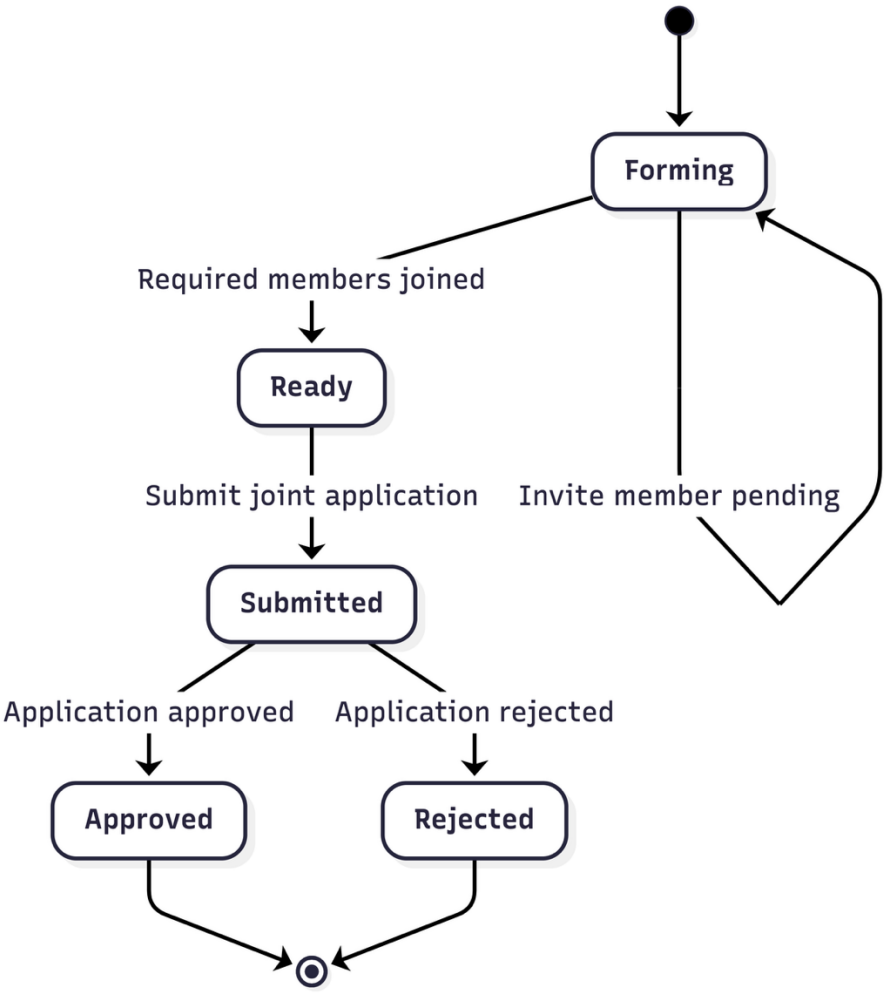


Figure 12 – State Diagram: Roommate Group.

The diagram in figure 12 represents the lifecycle of a roommate group in the system. initially, the group is in the forming state while members are being invited and accepted. once the required number of members has joined, the group moves to the ready state. after submitting a joint application, the group enters the submitted state, which can result in either approved or rejected outcomes. this diagram helps illustrate how the system controls group progression and enforces business rules during the roommate matching process.

3 SYSTEM ARCHITECTURE

3.1 Architecture Overview and Style Justification

The Roommate Matching System follows a layered architecture that separates user interaction, system logic, and data storage. This approach improves clarity, maintainability, and scalability of the system. It allows core features such as roommate matching, group formation, and application submission to be developed and managed independently while maintaining consistency across

3.2 Component Diagram

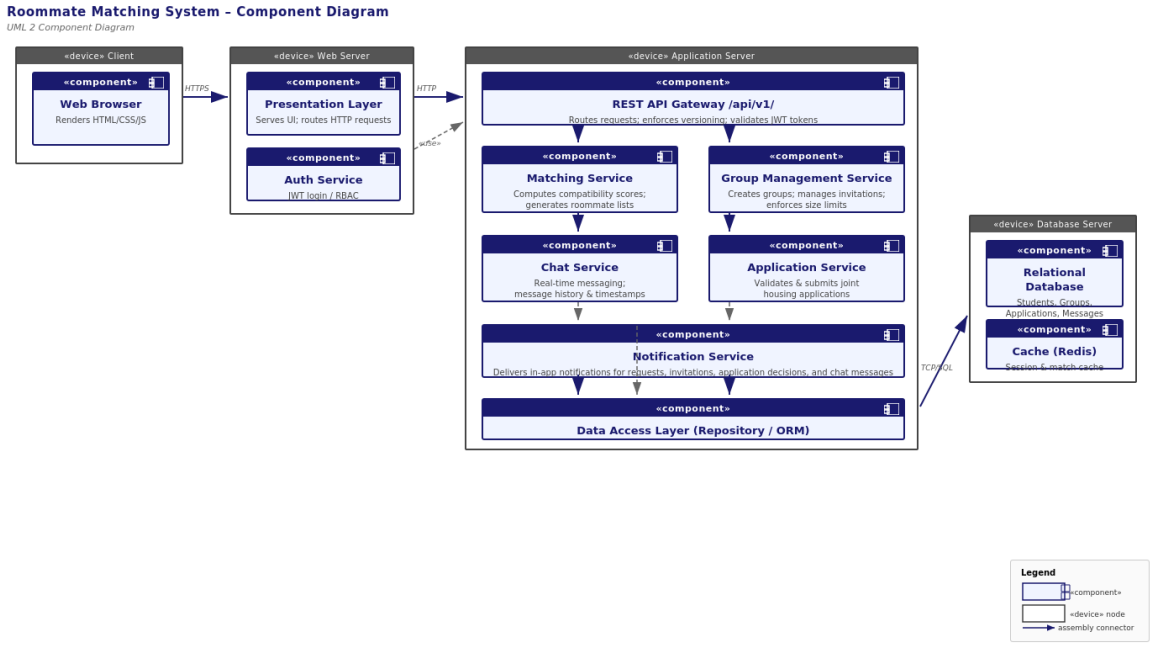


Figure 13 – Component Diagram.

This component diagram shows the main software components of the Roommate Matching System and how they interact. Users access the system through the web interface, which communicates with the matching, group management, and application processing components. All components rely on a shared database to store and retrieve system data, ensuring consistent and centralized data management.

3.3 Database Strategy

The system uses a relational database to store structured data such as student profiles, compatibility criteria, match results, roommate groups, and joint applications. The database is designed using normalized tables with primary and foreign key constraints to ensure data integrity and reduce redundancy. This strategy supports efficient querying, reliable transaction handling, and consistent data access across all system components.

3.4 Deployment Diagram

The deployment diagram illustrates the physical distribution of the Roommate Matching System across different computing nodes. Students and housing administrators access the system through a web browser on their devices. The web server hosts the presentation layer and forwards user requests to the application server, which exposes system functionality through a REST API. The application server coordinates backend services responsible for roommate matching, group management, and joint application processing. All services interact with a centralized database server that stores persistent system data. This deployment structure supports scalability, security, and efficient separation of system responsibilities.

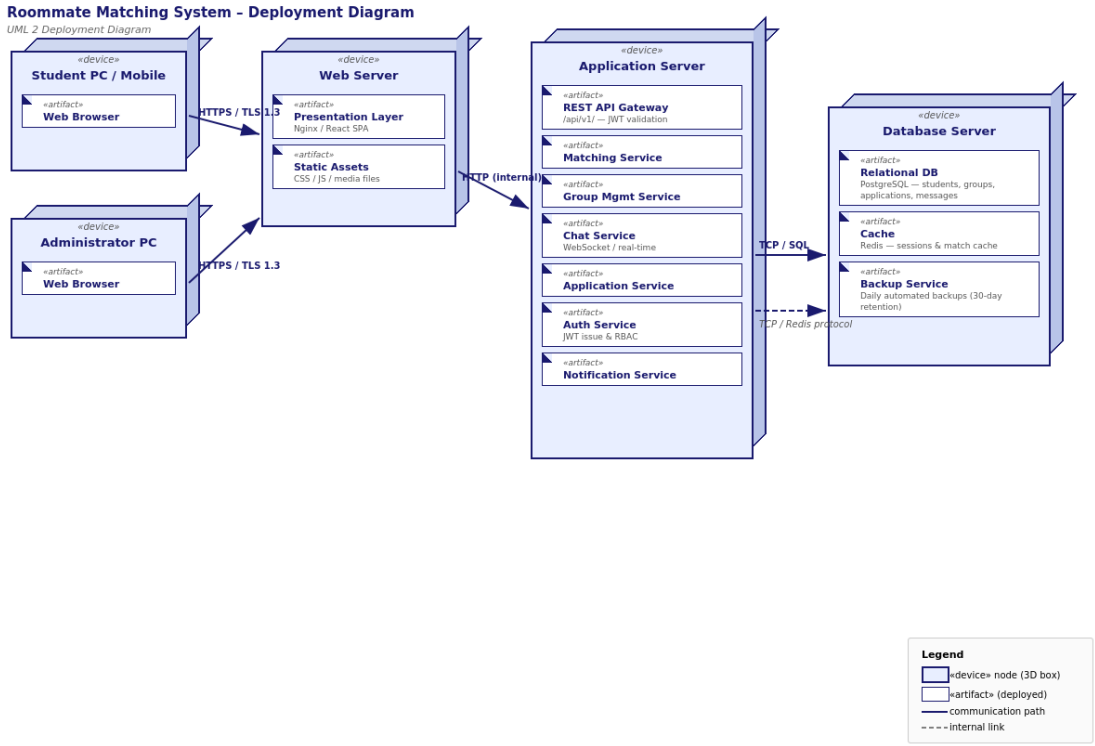


Figure 14 – Deployment Diagram.

3.5 Architecture Decision Records

Use of Layered Architecture: This approach separates concerns, improves maintainability, and allows each layer to evolve independently. It also aligns with standard web system design and supports future system extension.

Use of REST API for Communication: REST APIs provide a standardized and scalable communication mechanism. This decision enables clear client–server separation and allows the system to support multiple clients in the future.

Relational Database Selection: Relational databases ensure data consistency through constraints and relationships, which is essential for managing student profiles, roommate groups, and joint applications.

JWT Authentication: JWT-based authentication was chosen to provide stateless and secure user authentication, making it suitable for REST-based web applications.

4. API DESIGN

4.1 API Design Principles

The Roommate Matching System exposes its functionality through a RESTful API following these core principles:

1. Resource-based URLs:
   /students, /criteria, /matches, /groups, /applications

2. Standard HTTP methods:
   GET (retrieve), POST (create), PUT (update), DELETE (remove)

3. Stateless communication:
   Each request contains all necessary authentication and context information.

4. Security:
   – HTTPS-only communication
   – JWT tokens for secure access
   – Tokens transmitted via Authorization headers

5.Versioning:
API versioning is implemented using the URL path: /api/v1/...

6. Consistent data format:

All requests and responses use JSON format.

7.Filtering and pagination: Collection endpoints support filtering, sorting, and pagination where applicable.

4.2 Authentication Strategy

The system implements JWT (JSON Web Tokens) for stateless authentication:

– Login: User submits credentials → system validates → JWT token is returned

– Token structure: Contains user ID, role, and expiration time

– Authorization: Role-based access control (RBAC) with three roles:

– Student: Profile management, matching, group formation, application submission

– Housing Administrator: Application review and approval

– System Admin: User and system configuration management

This approach ensures secure access while supporting REST-based communication.

4.3 REST API Endpoint Specification

Core Endpoints for Roommate Matching

POST /criteria – Set compatibility criteria

GET /matches/{studentId} – Retrieve matching results

GET /matches/{studentId}/sorted – Retrieve filtered or sorted matches

Group Management

POST /groups – Create a roommate group

POST /groups/{id}/invite – Send group invitation

PUT /groups/{id}/response – Accept or decline invitation

GET /groups/{id} – View group details and status

Joint Application Processing

POST /applications – Submit joint application

GET /applications/{id} – View application status

User Management

POST /auth/login – Authenticate user

POST /auth/register – Register new student

GET /students/{id} – View student profile

PUT /students/{id} – Update student profile

4.4 Request and Response Examples

Example 1: Retrieve Matching Results

GET /api/v1/matches/42?minScore=0.7&page=1 Authorization: Bearer <token>

Response:

```
{
  "matches": [
    {
```

```
    "studentId": 58,
    "compatibilityScore": 0.85
  }
],
"total": 1,
"page": 1
}
```


Example 2: Submit Joint Application

POST /api/v1/applications Authorization: Bearer <token> Content-Type: application/json


```
{
  "groupId": 12,
  "preferredHousing": "Campus Residence A"
}
```


Response:


```
{
  "applicationId": 301,
  "status": "submitted"
}
```


4.5 Additional API Features

1. JSON-based request and response format
2. URL versioning (/api/v1/...)
3. Role-based access control for protected endpoints

4. Consistent HTTP status codes and error messages
5. Pagination, filtering, and sorting support on collection endpoints

CONCLUSION

This coursework presented the complete analysis and design of a Roommate Matching System for university students. The work addressed a genuine gap in student housing administration — the absence of a structured, digital process for roommate search, compatibility assessment, group formation, and joint application submission — and produced a comprehensive set of design artifacts to fill that gap.

Chapter 1 established the system scope through stakeholder analysis, user stories with acceptance criteria, and functional and non-functional requirements. The use case diagram and written scenario for joint application submission provided a clear picture of system boundaries and actor interactions. Chapter 2 modelled system behaviour using six complementary notations: BPMN diagrams captured the key business processes of roommate browsing, group formation, and application submission; DFDs at three levels traced data flow from student input through system processing to persistent storage; the ER diagram and data dictionary defined the relational data structure; the UML class diagram represented the static object model; sequence diagrams illustrated interaction flows for the three main scenarios; and the state diagram modelled the lifecycle of a roommate group from formation through to application outcome.

Chapter 3 defined the system architecture using a layered approach with a REST API gateway coordinating dedicated services for matching, group management, chat, application processing, authentication, and notifications. Proper UML 2 component and deployment diagrams specified the software components and their physical distribution across client, web server, application server, and database server nodes. Chapter 4 completed the design with a REST API specification covering all core endpoints, authentication strategy using JWT and role-based access control, and request and response examples.

The resulting design is internally consistent, traceable to the original requirements, and structured to support straightforward implementation using standard web technologies.

REFERENCES

1. Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley Professional.
2. Richardson, L., & Amundsen, M. (2021). *RESTful Web APIs: Services for a Changing World*. O'Reilly Media.
3. Object Management Group. (2017). *OMG Unified Modeling Language Specification, Version 2.5.1*. Retrieved from https://www.omg.org/spec/UML/2.5.1
4. Object Management Group. (2013). *Business Process Model and Notation (BPMN) Version 2.0.2*. Retrieved from https://www.omg.org/spec/BPMN/2.0.2
5. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson Education.
6. Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.
7. OpenAPI Initiative. (2021). *OpenAPI Specification Version 3.0.3*. Retrieved from https://spec.openapis.org/oas/v3.0.3
8. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.